Programmation Orientée Objet (POO) à destination de la robotique

Travaux Pratique : Capteurs pour robot mobile

Enseignant : Adam GOUGUET
Mail : adam.gouguet@imt-nord-europe.fr
Année : 2025-2026

1 Sujet du TP : Capteurs pour robot mobile

Contexte

Dans ce TP, vous allez travailler sur la programmation orientée objet (POO) appliquée aux capteurs d'un robot mobile. L'objectif est de comprendre comment des capteurs différents peuvent être modélisés et intégrés dans une simulation ou un robot réel, tout en restant modulaires et extensibles.

Chaque robot peut être équipé de plusieurs types de capteurs : distance (Lidar, Ultrason), position (GPS), mouvement (IMU, encodeurs), contact (Bumper), etc. Votre tâche est de concevoir une architecture POO permettant de gérer ces capteurs de manière cohérente.

Objectifs pédagogiques

- Comprendre le rôle de l'abstraction et du polymorphisme en POO.
- Savoir concevoir une architecture extensible pour gérer plusieurs capteurs.
- Relier les lectures des capteurs aux besoins d'un robot autonome.
- Préparer les bases pour implémenter des algorithmes de navigation et de perception.

Conseils et indices

- Chaque capteur est responsable de sa propre lecture et de sa représentation, mais ne doit pas gérer le robot ou l'environnement directement.
- Le robot possède une liste de capteurs, et interagit avec eux via l'interface commune.
- Vous pouvez vous inspirer des capteurs réels pour définir les paramètres : portée, fréquence, précision, bruit.
- La visualisation peut rester simple : traits, cercles ou texte, l'important est de montrer les informations de chaque capteur.

1.1 Description des capteurs à implémenter

Vous devez implémenter plusieurs capteurs inspirés de capteurs réels utilisés en robotique mobile. Chaque capteur possède un comportement physique différent, ce qui justifie une implé-

mentation spécifique tout en respectant une interface commune. Chaque capteur hérite d'une classe abstraite commune et doit fournir :

— une méthode `read(env)` retournant l'observation du capteur,
— optionnellement une méthode `draw(...)` pour l'affichage.

1. Capteur de distance (type Lidar)

Le capteur Lidar simule un télémètre qui envoie plusieurs rayons autour du robot afin de mesurer la distance aux obstacles. Chaque rayon correspond à une direction différente et retourne la distance jusqu'au premier obstacle rencontré, ou la portée maximale s'il n'y a rien.

Principe général Le capteur génère un ensemble d'angles répartis uniformément dans son champ de vision. Pour chaque angle :

— on calcule la direction du rayon à partir de l'orientation du robot,
— on teste l'intersection de ce rayon avec tous les obstacles,
— on conserve la plus petite distance trouvée,
— si aucune intersection n'est détectée, on garde la distance maximale.

Calcul de la direction Pour un angle θ donné :

$$dx = \cos(\theta), \quad dy = \sin(\theta)$$

Le point détecté à distance d est :

$$(x, y) = (x_r, y_r) + d(dx, dy)$$

Algorithme simplifié

```
pour chaque rayon:
    calculer son angle
    calculer sa direction
    distance_min = portée max

    pour chaque obstacle:
        calculer intersection rayon/obstacle
        si intersection:
            garder la plus petite distance

    stocker distance_min
```

Gestion des obstacles Chaque type d'obstacle possède sa propre méthode d'intersection avec un rayon :

— Cercle : résolution d'une équation quadratique pour trouver le point de contact.
— Rectangle : calcul des intersections avec les bords.
— Segment : résolution d'un système pour trouver l'intersection entre deux segments.

Cela permet d'ajouter facilement de nouveaux types d'obstacles sans modifier le capteur, grâce au polymorphisme.

Calcul d'intersection selon le type d'obstacle

Notations communes

— (ox, oy) : origine du rayon (position du robot)
— (dx, dy) : direction du rayon (vecteur unitaire)
— portée max : distance maximale mesurable
— t : distance le long du rayon telle que

$$P(t) = (ox, oy) + t(dx, dy)$$

— Une intersection valide doit toujours vérifier :

$$0 < t \leq \text{portée max}$$

1) Cercle Principe : on remplace l'équation paramétrique du rayon dans l'équation du cercle, ce qui donne une équation du second degré.

Variables spécifiques :
— (cx, cy) : centre du cercle
— r : rayon du cercle
— (fx, fy) : vecteur centre -> origine du rayon
— a, b, c : coefficients du polynôme quadratique
— discriminant : permet de savoir s'il existe une intersection
— t_1, t_2 : distances possibles d'intersection

```python
def intersection(self, ox, oy, dx, dy, max_range):
    fx = ox - self.x
    fy = oy - self.y

    a = dx*dx + dy*dy
    b = 2 * (fx*dx + fy*dy)
    c = fx*fx + fy*fy - self.rayon*self.rayon

    discriminant = b*b - 4*a*c
    if discriminant < 0:
        return None

    discriminant = math.sqrt(discriminant)
    t1 = (-b - discriminant) / (2*a)
    t2 = (-b + discriminant) / (2*a)

    for t in (t1, t2):
        if 0 < t <= max_range:
            return t

    return None
```

Interprétation :
— discriminant < 0 alors le rayon ne touche jamais le cercle
— deux solutions alors le rayon entre puis sort du cercle

— on garde la première intersection rencontrée

2) Rectangle On calcule les intervalles d'intersection du rayon avec les bandes verticales et horizontales, puis on garde leur intersection.

Variables spécifiques :

— t_1, t_2 : distances d'intersection avec les deux faces d'un axe

— $t_{\min}$: première intersection valide globale

— $t_{\max}$: sortie du rectangle

```
def intersection(self, ox, oy, dx, dy, max_range):
    t_min = 0
    t_max = max_range

    x_min = self.x
    x_max = self.x + self.largeur
    y_min = self.y
    y_max = self.y + self.hauteur

    for o, d, mn, mx in (
        (ox, dx, x_min, x_max),
        (oy, dy, y_min, y_max),
    ):
        if abs(d) < 1e-8:
            if o < mn or o > mx:
                return None
        else:
            t1 = (mn - o) / d
            t2 = (mx - o) / d
            t1, t2 = min(t1, t2), max(t1, t2)
            t_min = max(t_min, t1)
            t_max = min(t_max, t2)
            if t_min > t_max:
                return None

    if 0 < t_min <= max_range:
        return t_min

    return None
```

Interprétation :

— t_1 = entrée dans la bande

— t_2 = sortie de la bande

— on intersecte les intervalles X et Y

3) Segment Principe : on résout l'intersection entre deux segments paramétriques (le rayon et le segment obstacle).

Variables spécifiques :

— (sx, sy) : vecteur direction du segment

— det : déterminant du système (teste si parallèles)

— u : distance d'intersection le long du rayon
— t : position sur le segment (entre 0 et 1)

```
def intersection(self, ox, oy, dx, dy, max_range):
    ax, ay = self.x1, self.y1
    bx, by = self.x2, self.y2

    sx = bx - ax
    sy = by - ay

    det = dx * (-sy) - dy * (-sx)
    if abs(det) < 1e-8:
        return None # parallele

    inv_det = 1.0 / det

    u = ((ax - ox) * (-sy) - (ay - oy) * (-sx)) * inv_det
    t = ((dx) * (ay - oy) - (dy) * (ax - ox)) * inv_det

    if 0 < u <= max_range and 0 <= t <= 1:
        return u

    return None
```

Interprétation :
— u indique à quelle distance le rayon touche le segment
— t indique où sur le segment se trouve l'impact
— $t \in [0, 1]$ garantit que l'impact est bien sur le segment

Structure logicielle
— Classe `Lidar` : gère la génération des rayons et la mesure.
— Classe abstraite `Obstacle` : définit l'interface commune.
— Sous-classes d'obstacles : implémentent leurs propres calculs d'intersection.
Cette architecture orientée objet rend le système modulaire, extensible et facile à maintenir.

2. GPS (position globale)

Fournit la position absolue du robot dans l'environnement sous forme de coordonnées (x, y). Peut être bruité et avoir une fréquence de mise à jour limitée.

Caractéristiques :
— Coordonnées x, y
— Possibilité d'ajouter du bruit
— Fréquence de mise à jour plus lente qu'un capteur local

> On ajoute du bruit pour simuler le bruit réel que peut avoir un GPS.

Pseudo-logique :

```
mesure_x = robot.x + bruit()
mesure_y = robot.y + bruit()
```

3. IMU (centrale inertielle)

Mesure l'état dynamique du robot, comme son orientation, sa vitesse angulaire ou son accélération. Utile pour estimer le mouvement et la stabilité.

Mesures possibles :

— orientation
— vitesse angulaire
— accélération

Pseudo-logique :

```
omega = vitesse_angulaire_robot + bruit
acc = vitesse_actuelle - vitesse_precedente
```

4. Bumper (capteur de contact)

Détecte si le robot entre en collision avec un obstacle. Retourne généralement une information binaire indiquant contact ou non.

Sortie :

— 0 -> pas de collision
— 1 -> collision

Pseudo-logique :

```
pour obstacle:
    si collision(robot, obstacle):
        return 1
return 0
```

5. Encodeurs de roues

Mesurent la rotation des roues pour estimer la distance parcourue et la vitesse du robot. Ils permettent de suivre le déplacement sans capteur externe.

Mesures :

— distance parcourue
— vitesse estimée
— enregistre la trajectoire prise par le robot

Indice math :

$$distance = \sqrt{(x - x_{old})^2 + (y - y_{old})^2}$$

Pseudo-logique :

```
dx = x - last_x
dy = y - last_y
distance = sqrt(dx^2 + dy^2)
mettre_a_jour last_x last_y
```

But global

Vous devez concevoir votre architecture de sorte que :

— tous les capteurs puissent être utilisés de manière interchangeable,
— un robot puisse en posséder plusieurs simultanément,
— l'ajout d'un nouveau capteur ne nécessite pas de modifier les capteurs existants ou le reste du projet.

Travail demandé

1. **Classe abstraite Capteur** :
   — Définir une interface commune pour tous les capteurs.
   — Préciser les méthodes essentielles que tout capteur doit fournir (ex : lecture des données).
2. **Implémentations concrètes** :
   — Créer plusieurs types de capteurs héritant de la classe abstraite (ex : Lidar, GPS, IMU, Bumper).
   — Chaque capteur doit encapsuler sa logique propre et pouvoir fournir une lecture normalisée.
   — Réfléchir à la manière de gérer les paramètres spécifiques à chaque capteur (portée, bruit, fréquence, etc.).
3. **Visualisation et interaction** :
   — Prévoir une méthode pour que chaque capteur puisse être représenté dans la simulation (texte, traits, vecteurs, halo...).
   — La visualisation doit être modulable : on peut activer/désactiver ou ajouter/retirer un capteur sans modifier le reste du code.
4. **Extension et modularité** :
   — Votre architecture doit permettre d'ajouter facilement de nouveaux capteurs sans toucher aux capteurs existants.
   — Penser à l'utilisation du polymorphisme et des classes abstraites pour rendre votre code flexible.